

Practical 12

Producer-consumer with POSIX semaphores + pthreads

```
gcc -Wall -Wextra -pthread producer_consumer.c -o producer_consumer
time ./producer_consumer
```

Change BUFFER_SIZE to values such as 4, 16, 64, and compare runtime. The producer waits on empty_slots; the consumer waits on full_slots; the mutex protects in, out, and the buffer.

Intentional deadlock

```
gcc -Wall -Wextra -pthread deadlock_demo.c -o deadlock_demo
timeout 5s ./deadlock_demo
```

Two threads acquire resources in opposite order:

- T1: A -> B
- T2: B -> A

The four Coffman conditions are:

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Deadlock prevention using resource ordering

```
gcc -Wall -Wextra -pthread deadlock_prevention.c -o deadlock_prevention
./deadlock_prevention
```

Both threads acquire A before B. This removes circular wait, so the deadlock cannot form.

Deadlock_demo.c-

```
#include <pthread.h> #include <stdio.h> #include <unistd.h>
```

```
static pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER; static pthread_mutex_t B =
PTHREAD_MUTEX_INITIALIZER;
```

```
static void *thread1(void *arg) { (void)arg; pthread_mutex_lock(&A); printf("T1 locked A\n"); sleep(1); printf("T1 waiting for B...\n"); pthread_mutex_lock(&B); printf("T1 locked B\n"); pthread_mutex_unlock(&B); pthread_mutex_unlock(&A); return NULL; }
```

```
static void *thread2(void *arg) { (void)arg; pthread_mutex_lock(&B); printf("T2 locked B\n"); sleep(1); printf("T2 waiting for A...\n"); pthread_mutex_lock(&A); printf("T2 locked A\n"); pthread_mutex_unlock(&A); pthread_mutex_unlock(&B); return NULL; }
```

```
int main(void) { pthread_t t1, t2; puts("Intentional deadlock demo. It will not terminate normally."); puts("Run with: timeout 5s ./deadlock_demo"); pthread_create(&t1, NULL, thread1, NULL); pthread_create(&t2, NULL, thread2, NULL); pthread_join(t1, NULL); pthread_join(t2, NULL); return 0; }
```

Deadlock_prevention.c-

```
#include <pthread.h> #include <stdio.h> #include <unistd.h>
```

```
static pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER; static pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
```

```
/* Resource ordering: every thread locks A before B. */ static void *worker(void *arg) { long id = (long)arg;
```

```
pthread_mutex_lock(&A);  
printf("T%d locked A\n", id);  
usleep(100000);
```

```
pthread_mutex_lock(&B);  
printf("T%d locked B\n", id);  
printf("T%d using both resources\n", id);
```

```
pthread_mutex_unlock(&B);  
pthread_mutex_unlock(&A);  
return NULL;
```

```
}
```

```
int main(void) { pthread_t t1, t2;
```

```
pthread_create(&t1, NULL, worker, (void *)1);  
pthread_create(&t2, NULL, worker, (void *)2);
```

```
pthread_join(t1, NULL);
```

```
pthread_join(t2, NULL);
```

```
puts("Completed without deadlock because resource ordering is  
consistent.");  
pthread_mutex_destroy(&A);  
pthread_mutex_destroy(&B);  
return 0;  
  
}
```

Producer_consumer.c-

```
#define _POSIX_C_SOURCE 200809L #include <pthread.h> #include <semaphore.h>  
#include <stdio.h> #include <stdlib.h> #include <unistd.h>  
  
#define ITEMS 100000 #define BUFFER_SIZE 16  
  
static int buffer[BUFFER_SIZE]; static int in = 0, out = 0; static sem_t empty_slots,  
full_slots; static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
  
static void *producer(void *arg) { (void)arg; for (int i = 1; i <= ITEMS; i++)  
{ sem_wait(&empty_slots); pthread_mutex_lock(&mutex); buffer[in] = i; in = (in + 1) %  
BUFFER_SIZE; pthread_mutex_unlock(&mutex); sem_post(&full_slots); } return NULL; }  
  
static void *consumer(void *arg) { (void)arg; long sum = 0; for (int i = 1; i <= ITEMS; i++)  
{ sem_wait(&full_slots); pthread_mutex_lock(&mutex); int value = buffer[out]; out = (out  
+ 1) % BUFFER_SIZE; pthread_mutex_unlock(&mutex); sem_post(&empty_slots); sum  
+= value; } printf("Consumer received %d items, checksum=%ld\n", ITEMS, sum); return  
NULL; }  
  
int main(void) { pthread_t p, c;  
  
sem_init(&empty_slots, 0, BUFFER_SIZE);  
sem_init(&full_slots, 0, 0);  
  
pthread_create(&p, NULL, producer, NULL);  
pthread_create(&c, NULL, consumer, NULL);  
  
pthread_join(p, NULL);  
pthread_join(c, NULL);  
  
sem_destroy(&empty_slots);  
sem_destroy(&full_slots);
```

```
pthread_mutex_destroy(&mutex);  
return 0;
```

```
}
```